NEWMAN CLI & ADVANCED API TESTING

Complete Beginner-Friendly Notes

Run Postman collections from the terminal • Generate HTML reports • PATCH resources • Validate JSON Schema

Built from the supplied Flow(6).txt lesson, with code walkthroughs, examples, troubleshooting, QA scenarios, and best practices.

API Testing Learning Series

How to Use These Notes

This lesson assumes you have already created a Postman collection and exported two files: a collection JSON file and an environment JSON file. The sample names used throughout are “Booking API.postman_collection.json” and “Booking API.postman_environment.json”.

Read the Newman sections first, then practise PATCH, and finish with JSON Schema validation. Run the commands in a test environment and use only records that you are authorized to create or modify.

Source boundary: The practical commands and sample output below follow Flow(6). Additional explanations, safe execution patterns, and test-design recommendations are labelled as guidance so they are not confused with the exact wording of the lesson.

Learning outcomes

- Explain what Newman is and why it is used with Postman collections.
- Install Newman and confirm that the command is available.
- Run a collection with its matching environment JSON file.
- Understand the terminal summary and diagnose the filename-with-spaces error.
- Generate standard and extended HTML reports.
- Send a PATCH request to update only selected booking fields.
- Validate a response against a JSON Schema.
- Apply Newman in repeatable regression and CI smoke-test workflows.

1. What Is Newman?

Newman is the command-line collection runner for Postman. It executes requests, collection scripts, and tests from a Postman collection without requiring you to click Send for every request in the Postman desktop application.

Think of Postman as the place where you design and organize API requests. Newman is the tool that runs those saved requests from a terminal or an automated pipeline.

Why do QA engineers use Newman?

- Run the same API regression suite repeatedly.
- Execute API checks after a new build or deployment.
- Run tests on a machine or CI runner without manually opening Postman.
- Produce execution output and HTML reports that can be shared with a team.
- Make API validation part of a build-and-release process.

How Newman fits into the workflow

- Create and test requests in Postman.
- Group related requests into a collection.
- Create an environment containing values such as base URL, first name, booking ID, and token.
- Export the collection and environment as JSON files.
- Run both files through Newman.
- Review test results and HTML reports; fix failed requests or assertions.

2. Prerequisites and Installation

Newman is distributed through npm, the Node.js package manager. Install Node.js and npm first. On macOS, open Terminal; on Windows, use PowerShell or Command Prompt.

Step 1 — Check Node.js and npm

Terminal commands

```
node --version
npm --version
```

The commands print the installed versions. If either command is not recognized, install Node.js using an approved installation method and reopen the terminal.

Step 2 — Install Newman globally

Command from the lesson

```
npm install -g newman
```

Meaning of each part:

Part	Meaning
npm	Runs the Node.js package manager.
install	Downloads and installs a package.
-g	Installs the command globally so it can be invoked from the terminal.
newman	The package being installed.

Step 3 — Verify Newman

Terminal command

```
newman --version
```

A version number confirms that the Newman command is available in the current shell. If it is not found, reopen the terminal and check whether the npm global executable directory is on PATH.

Practical note: In a team or CI environment, pin and document the Newman version instead of allowing different machines to run different versions unknowingly.

3. Export the Collection and Environment

The lesson uses two exported JSON files. Both are needed because the collection describes the requests and scripts, while the environment supplies variable values referenced by those requests.

File	What it contains	Example values
Collection JSON	Requests, methods, URLs, headers, bodies, folders, scripts, and tests.	GET Booking Ids, Create Booking, Update Booking, PATCH request.
Environment JSON	Environment-scoped variables selected for the run.	baseUrl, fname, b_id, b_token.

A variable such as {{b_id}} is a placeholder. At runtime, Postman/Newman resolves it using the selected environment or another applicable variable scope.

Security: Environment exports can contain tokens, credentials, or other sensitive values. Do not commit real secrets to a public repository or attach an unredacted environment export to a ticket. Use protected CI secrets or a controlled secret store for sensitive values.

4. Run a Collection from the Terminal

Basic Newman syntax

```
newman run <collection_file> -e <environment_file>
```

Explanation:

- newman run tells Newman to execute a collection.
- <collection_file> is the path to the exported Postman collection JSON.
- -e tells Newman which environment file to load.
- <environment_file> is the path to the exported environment JSON.

Command for the Booking API files

Recommended command when filenames contain spaces

```
newman run "Booking API.postman_collection.json" -e "Booking API.postman_environment.json"
```

The quotation marks make the shell treat each filename as one argument. The command assumes both files are in the current working directory.

Why the first command failed

```
newman run Booking API.postman_collection.json -e Booking API.postman_environment.json
```

The terminal interprets spaces as separators between command arguments. Newman may therefore receive “Booking” as the filename instead of the complete filename. The lesson output shows:

```
error: could not load environment
ENOENT: no such file or directory, open 'Booking'
```

ENOENT means “Error NO ENTry”: the requested file or directory could not be found at the path Newman tried to open. In this example, the root cause is the unquoted filename containing spaces, not necessarily a missing export.

Two valid ways to handle spaces

```
newman run "Booking API.postman_collection.json" -e "Booking API.postman_environment.json"

# macOS/Linux alternative: escape each space
newman run Booking\ API.postman_collection.json -e Booking\ API.postman_environment.json
```

Confirm that the files are in the current folder

```
pwd
ls -l
```

pwd prints the current directory. ls -l lists the files in that directory. If the JSON files are stored elsewhere, use their correct relative or absolute paths, or change directory with cd before running Newman.

Troubleshooting sequence: When Newman reports ENOENT, check the spelling, extension, current directory, path, and quoting/escaping before reinstalling Newman.

5. Understand the Newman Terminal Output

The lesson shows Newman executing the Booking API collection and printing each request, response status, response size, response time, and test result.

```
Booking API

❏ Positive Endpoints
↳ GET Booking Ids
 GET https://restful-booker.herokuapp.com/booking [200 OK, 5.1kB, 1861ms]
 ✓ Verify Status Code is 200

↳ Create Booking
 POST https://restful-booker.herokuapp.com/booking [200 OK, 953B, 263ms]
 ✓ Verify Status Code is 200
 ✓ Verify Body matches string
 ✓ Content-Type is present
 ✓ Response time is less than 2000ms

❏ Negative Scenario
↳ GET Booking Details Invalid ID
 GET https://restful-booker.herokuapp.com/booking/48746 [404 Not Found, 751B, 288ms]
 ✓ Status code is 404
```

Output item	What it tells you
Collection name	The collection Newman is running.
Folder name	The group of requests being executed.
HTTP method and URL	Which API endpoint Newman called.
200 OK / 404 Not Found / 400 Bad Request	The HTTP response status returned by the API.
5.1kB / 953B	Approximate response payload size shown by Newman.
1861ms / 263ms	Time taken for that individual response.
✓ test name	A Postman test assertion passed.
failed count	How many requests or assertions failed in the run summary.

Understand the final run summary

Metric in the lesson	Sample value	Meaning
Iterations	1	The collection iteration ran once.
Requests	9 executed, 0 failed	Nine requests were executed and none failed at the request level.
Test scripts	8 executed	Eight request-level test scripts ran.
Assertions	15 executed, 0 failed	All 15 assertions passed.
Total run duration	4.5s	Approximate duration of the complete collection run.
Total data received	5.19kB	Approximate total response data received.
Average response time	490ms	Average response time across the sample run.

Important distinction: One successful Newman run is evidence that the checks passed in that run. It is not proof that the API is always healthy or that it can handle production load. Response times vary with network, server load, and environment.

The deprecation warning in the sample

The source output also displays a Node.js DeprecationWarning related to fs.F_OK. A deprecation warning indicates that some code is using an API marked as deprecated. In the shown session, the warning did not prevent the later successful run. Treat it separately from the ENOENT file-loading error.

6. Generate a Standard HTML Report

A terminal log is useful while running commands. An HTML report is easier to browse, archive, and share with a QA team or release reviewer.

Step 1 — Install the HTML reporter

Command shown in the lesson

```
npm install -g newman-reporter-html
```

This installs the standard HTML reporter package globally. Use the same package installation context that Newman can resolve on the machine where you execute the collection.

Step 2 — Run Newman with the HTML reporter

```
newman run "Booking API.postman_collection.json" -e "Booking API.postman_environment.json" -r html
```

The -r option selects a reporter. Here, html tells Newman to generate an HTML report. The lesson states that a folder named newman is created and the report is written there by default.

Step 3 — Open and review the report

- Open the generated newman folder.
- Locate the HTML report file.
- Open it in a browser.
- Review passed and failed requests, assertions, response details, and timing information.
- Archive the report with the build number or test execution date if your team needs traceability.

7. Generate an Advanced HTML Report

The lesson uses the Newman HTML Extra reporter to create a more detailed report layout.

Step 1 — Install the reporter

Command shown in the lesson

```
npm install -g newman-reporter-htmlextra
```

Step 2 — Execute with CLI and htmlextra reporters

```
newman run "Booking API.postman_collection.json" -e "Booking API.postman_environment.json" -r cli,htmlextra
```

The comma-separated reporter list asks Newman to produce normal command-line output and an HTML Extra report in the same run.

Standard HTML vs HTML Extra

Reporter	Purpose
cli	Displays execution progress and test results in the terminal.
html	Creates the standard HTML report.

Reporter	Purpose
htmlextra	Creates an extended HTML report with a richer report presentation.

Version and installation note: The source lesson demonstrates global npm installation. In a controlled project or CI pipeline, document the reporter package version and ensure the package is installed where the Newman process can find it. If a reporter is reported as missing, inspect the npm global installation and PATH before changing the collection.

8. PATCH Request — Partial Update

What is PATCH?

PATCH is an HTTP method used to partially modify an existing resource. Instead of sending every field of a booking, the client sends only the fields it wants to change, when the API supports partial updates.

Example: a booking currently contains firstname, lastname, totalprice, depositpaid, bookingdates, and additionalneeds. If only the guest name changes, a PATCH request can contain just firstname and lastname.

Request configuration from the lesson

Setting	Value
Folder	Positive Endpoints
Request name	Partial Update Booking
Method	PATCH
URL	https://restful-booker.herokuapp.com/booking/{{b_id}}
Body mode	raw → JSON
Authentication	Use the token cookie required by the demo API, as configured for the other protected update requests.

Example PATCH body

```
{
  "firstname": "Abhishek",
  "lastname": "Biradar"
}
```

Explain the URL

The base path /booking identifies the booking resource. {{b_id}} is an environment variable containing the booking ID. Newman resolves the variable before sending the request. For example, if b_id is 351, the final path becomes /booking/351.

Execution order

The lesson says to execute Create Booking → Token Generator → Partial Update Booking. This order matters because the create request obtains the booking ID and the token request obtains the authentication token.

- Create Booking: creates a booking and stores the returned booking ID in the environment variable b_id, if the request script is configured to do so.
- Token Generator: authenticates and stores the returned token in b_token, if its response script is configured to do so.

• Partial Update Booking: uses {{b_id}} in the URL and the authentication token in the cookie header.

Verify the update: After PATCH, send GET /booking/{{b_id}} and assert that firstname and lastname have the new values. Also verify that fields not included in the PATCH body remain unchanged, if that is part of the API contract.

PATCH vs PUT

PATCH	PUT
Usually changes only the fields supplied in the request.	Usually replaces the target resource representation with the supplied representation.
Useful for a small edit such as changing a name.	Useful when the client intends to provide the complete updated representation.
Exact behavior depends on the API contract.	Exact behavior and required fields depend on the API contract.

9. JSON Schema Validation

What is a JSON Schema?

A JSON Schema is a contract that describes the expected structure and data types of a JSON document. It can define whether the response must be an object, which properties should exist, and whether each property should be a string, integer, boolean, array, or nested object.

In API testing, schema validation checks the response shape. It helps detect breaking changes such as a field changing from a number to a string or a required property disappearing.

Workflow from the lesson

• Send GET Booking Details and inspect its JSON response.
• Copy a representative response.
• Generate a JSON Schema from that response using a schema generator.
• Review the generated schema rather than blindly trusting it.
• Paste the schema into the request Scripts → Post-response area.
• Call the JSON Schema assertion to compare the actual response with the expected schema.

Review before use: A schema generated from one response may be too permissive or may accidentally encode one example rather than the real API contract. Check field types, required fields, nullable values, optional fields, and whether additional properties are allowed.

Expected schema from the lesson

```
const expectedJSONSchema = {
 "type": "object",
 "properties": {
  "firstname": { "type": "string" },
  "lastname": { "type": "string" },
  "totalprice": { "type": "integer" },
  "depositpaid": { "type": "boolean" },
  "bookingdates": {
   "type": "object",
   "properties": {
    "checkin": { "type": "string" },
    "checkout": { "type": "string" }
   },
   "required": ["checkin", "checkout"]
  },
  "additionalneeds": { "type": "string" }
 },
 "required": [
  "firstname",
  "lastname",
  "totalprice",
  "depositpaid",
  "bookingdates",
  "additionalneeds"
 ]
};
```

Line-by-line explanation

Schema keyword	Meaning in this example
const expectedJSONSchema	Declares a JavaScript constant that stores the expected response structure.
type: object	The complete response must be a JSON object.
properties	Lists the properties that the response object can contain and their expected definitions.
firstname / lastname: string	Both guest names must be JSON strings.
totalprice: integer	The price must be a whole number, not a quoted string or decimal under this schema.
depositpaid: boolean	The deposit flag must be true or false.
bookingdates: object	Booking dates are grouped inside a nested JSON object.
checkin / checkout: string	Both date fields must be strings in this schema.
required	Lists properties that must be present at the relevant object level.

Notice that bookingdates has its own required list. That makes checkin and checkout mandatory inside bookingdates. The top-level required list makes firstname, lastname, totalprice, depositpaid, bookingdates, and additionalneeds mandatory in the response.

Run the schema assertion

```
pm.test("Verify JSON Schema", function () {
 pm.response.to.have.jsonSchema(expectedJSONSchema);
});
```

Explain the assertion

- pm.test(name, function) creates a named test that appears in Postman/Newman results.
- The name Verify JSON Schema makes the test result understandable.
- pm.response refers to the response returned by the API.
- to.have.jsonSchema(expectedJSONSchema) checks whether the response body conforms to the supplied schema.
- If the response violates a defined type or required property, the assertion fails.

What this schema does not validate yet

- It does not validate that checkin and checkout are valid calendar dates; it only checks that they are strings.
- It does not check that checkout occurs after checkin.
- It does not check that totalprice is greater than zero.
- It does not reject additional response properties because additionalProperties is not set to false.
- It does not prove the returned values are correct for the particular booking; use separate value assertions for that.

Combine schema validation with business assertions

```
const body = pm.response.json();

pm.test("Response matches booking schema", function () {
  pm.response.to.have.jsonSchema(expectedJSONSchema);
});

pm.test("Booking price is positive", function () {
  pm.expect(body.totalprice).to.be.above(0);
});

pm.test("Check-in date is present", function () {
  pm.expect(body.bookingdates.checkin).to.be.a("string").and.not.empty;
});
```

The schema test validates structure and types. The other tests validate business conditions and specific values. A strong API test suite often needs both.

10. Real-Time QA Scenarios

Scenario A — Regression test after a deployment

A developer deploys a new Booking API build to the QA environment. The QA engineer runs the exported collection with the QA environment file using Newman. A failed status assertion or schema test blocks the release candidate from moving to the next stage until the failure is understood.

```
newman run "Booking API.postman_collection.json" -e "Booking QA.postman_environment.json" -r cli,htmlextra
```

Scenario B — Guest updates only their name

A customer contacts support to correct a spelling mistake in the guest name. The tester creates a test booking, obtains a token, PATCHes firstname and lastname, and then GETs the same booking ID. The test checks that the names changed and the booking dates and price were not unintentionally modified.

Scenario C — Third-party API contract changes

An upstream provider changes totalprice from a number to a string. A schema assertion expecting an integer fails immediately. The team can investigate whether the change is a defect, a documented contract update, or an incompatible release before the application consumes incorrect data.

Scenario D — Release evidence for a QA lead

After a regression run, Newman produces an HTML report. The QA engineer attaches the report to the build or test execution record, making it easier to see the tested requests, assertion outcomes, and response timings.

Scenario E — Detecting slow API responses

A response-time assertion can flag an endpoint that exceeds an agreed threshold. This is a useful smoke check, but it is not a substitute for load testing: Newman response times from a small collection do not model large concurrent user traffic.

11. Common Errors and Troubleshooting

Problem	Likely cause	What to check
ENOENT: could not load environment	Incorrect path, missing file, or unquoted filename with spaces.	Use quotes; verify pwd and ls -l; check the exact filename and extension.
newman: command not found	Newman is not installed or npm global bin is not on PATH.	Run npm install -g newman; reopen terminal; inspect npm global bin path.
Could not load collection	Collection path is wrong or JSON export is invalid.	Confirm the file exists and re-export a valid collection JSON.
Environment variable resolves empty	Wrong environment selected, variable disabled, or wrong key name.	Check environment file, exact spelling, enabled state, and variable scope.
401/403 on PATCH	Token missing, invalid, expired, or not sent in expected format.	Run Token Generator and verify Cookie header/token variable and API contract.
Schema assertion fails	Response has a different structure/type or schema is too strict.	Inspect actual JSON and compare property names, types, required fields, nullability.
Reporter not found	Reporter package is not installed where Newman can resolve it.	Check npm global package installation and PATH; install compatible reporter package.
HTML report not visible	Looking in the wrong output directory or report export location.	Inspect Newman output and the generated newman folder; confirm reporter ran.

12. Best Practices

- Keep collection and environment files together in a clearly named project folder, but keep secret-bearing environment values out of public source control.
- Always quote file paths that contain spaces. Use relative paths only when you know the current working directory.
- Use separate environments for local, QA, staging, and production. Do not accidentally run destructive requests against production.
- Keep request order intentional. Create resources before reading/updating them, obtain authentication before protected operations, and delete test data last.
- Keep one-purpose requests. A cURL-imported request that creates another booking should not be mixed into every regression run unless its extra side effect is intentional.

- Use dynamic IDs captured from responses rather than hard-coded IDs that may disappear or belong to another tester.
- Validate status, response structure, key values, and business rules. Do not rely only on a green status code.
- Use JSON Schema for contract shape and separate assertions for business correctness such as valid date ranges and positive totals.
- Keep response-time thresholds realistic for the target environment. Avoid treating one Newman run as a formal performance test.
- Generate an HTML report for traceability and keep the report associated with the commit/build/environment that was tested.
- Pin and document Node.js, Newman, and reporter versions in team automation to reduce “works on my machine” differences.
- Use a dedicated test account and data. Clean up test records safely and avoid deleting records created by other users.
- When a test fails, capture the response body and status for diagnosis, but redact tokens, passwords, personal information, and sensitive data from logs/reports.
- Review generated JSON Schemas. Avoid blindly copying schemas from a single sample response into a long-term contract test.
- Keep negative tests separate from normal success paths when they intentionally produce 400 or 404 responses. A negative test passes when the expected error behavior is verified.

13. Suggested End-to-End Execution Sequence

For the Booking API workflow taught across the supplied lesson, a sensible sequence is:

- GET Booking Ids (optional discovery check).
- Create Booking and capture b_id from the response.
- Token Generator and capture b_token from the response.
- GET Booking Details using {{b_id}} and validate response values/schema.
- PATCH Partial Update Booking using {{b_id}} and the token.
- GET Booking Details again to verify changed and unchanged fields.
- Run any PUT update test if it is part of the suite.
- DELETE the test booking at the end, and verify the expected deletion behavior.
- Run negative tests such as invalid booking ID and malformed JSON as independent checks.

Execution design: Negative tests do not need to be placed in the middle of a create-update-delete dependency chain. Keep them independent where possible so a failure in one test does not prevent unrelated checks from running.

14. Interview Questions and Quick Answers

What is Newman? Newman is the command-line runner for Postman collections. It runs saved requests and tests from a terminal or CI system.

Why do we need an environment file? It supplies variable values used by requests, such as base URL, booking ID, and authentication token.

What does -e mean? It tells Newman to load the specified environment JSON file.

Why did the filename command fail with ENOENT? The filenames contained spaces and were not quoted or escaped, so the shell split the filename into separate arguments.

What does -r html do? It asks Newman to use the standard HTML reporter.

What does -r cli,htmlextra do? It requests CLI output plus the HTML Extra reporter.

What is PATCH? PATCH partially updates a resource by sending the fields intended to change, when supported by the API.

What is JSON Schema validation? It verifies that a JSON response follows an expected structure, required-property set, and data types.

Does schema validation prove business correctness? No. It validates structure and types. Additional assertions are needed for correct values and business rules.

Is Newman a load-testing tool? No. It can perform basic response-time checks, but dedicated load-testing tools are needed for concurrency and capacity testing.

15. Hands-On Practice Exercises

- Install Newman and record the output of newman --version.
- Run the Booking API collection with the exported environment file using quoted filenames.
- Intentionally run the command without quotes to understand the filename-with-spaces error, then correct it.
- Install the standard HTML reporter and open the generated report.
- Install HTML Extra and generate a report using -r cli,htmlextra.
- Create a booking, capture its ID, obtain a token, and PATCH only firstname and lastname.
- GET the booking after PATCH and verify both updated and unchanged values.
- Add the JSON Schema assertion to GET Booking Details and run it through both Postman and Newman.
- Break one expected schema type in a test copy (for example, expect totalprice to be a string) and observe the assertion failure.
- Run the negative tests independently and confirm that the expected 404 and 400 outcomes are treated as passing tests.

16. Final Revision Checklist

- ☐ Node.js and npm are installed.
- ☐ Newman is installed and its version is verified.
- ☐ Collection and environment JSON files are exported.
- ☐ Newman command uses the correct file paths and quoted filenames.
- ☐ The terminal output and summary metrics are understood.
- ☐ Standard and HTML Extra reports can be generated.
- ☐ PATCH uses the correct booking ID and authentication token.
- ☐ GET after PATCH confirms the update.
- ☐ JSON Schema validates response structure and types.
- ☐ Business-rule assertions cover values not guaranteed by the schema.
- ☐ Secrets are protected and test data is cleaned up safely.

17. Source Coverage

These notes are based on the supplied Flow(6).txt lesson. The source lesson demonstrates Newman installation, collection execution with an environment file, a filename-with-spaces error and corrected command, sample execution output, standard and HTML Extra reporters, a PATCH request, and JSON Schema validation.

Additional explanations in this document expand the meaning of the commands and code, provide troubleshooting guidance, and add practical QA scenarios and best practices. Where a behavior depends on the API contract or installed package version, verify it against the version and documentation used by your project.